

MPEdit: SDEdit-Style Path Editing for Motion Planning Diffusion

Danny Broyd

Abstract

Diffusion-based motion planners can produce high-quality, collision-free trajectories, but they plan from scratch each time, discarding any prior solution. This report presents **MPEdit** (Motion Planning Editing), a novel framework that adapts the *SDEdit* image-editing paradigm to robot motion planning. Given an existing trajectory and a modified obstacle map, MPEdit adds controlled noise to the trajectory and denoises under an updated collision-cost guide, yielding a new collision-free path that stays close to the original. The same mechanism enables a *sketch-to-path* mode in which a rough, hand-drawn path is refined into a feasible trajectory. This project is built on Motion Planning Diffusion (MPD). The report evaluates MPEdit in both 2-D point-mass and 7-DOF Panda arm environments. Experiments show that MPEdit achieves competitive success rates while producing paths that are significantly more faithful to the original trajectory than replanning from scratch, at competitive times to classic motion planning algorithms and much faster than MPD inference.

1 Introduction

Classical motion planning algorithms such as RRT [4] can reliably find collision-free paths, but every invocation starts from scratch. When the environment changes only slightly, for example, a single obstacle is added or removed, re-running a planner wastes the information already embedded in the previous solution.

Recently, *diffusion models* have been applied to motion planning [2]. Motion Planning Diffusion (MPD) [1] trains a denoising diffusion model on a dataset of collision-free trajectories represented as B-spline control points. At inference time, cost guidance steers the denoising process away from collisions, producing smooth, high-quality paths. However, in the case of an environment change, MPD also generates trajectories from pure noise, ignoring any existing solution.

In image generation, *SDEdit* [5] showed that one can *edit* an existing image by adding noise at an intermediate diffusion step and then denoising. The amount of noise controls the trade-off between fidelity to the input and the model’s ability to “fix” the image.

This project combines these two ideas into **MPEdit**, introducing two key capabilities:

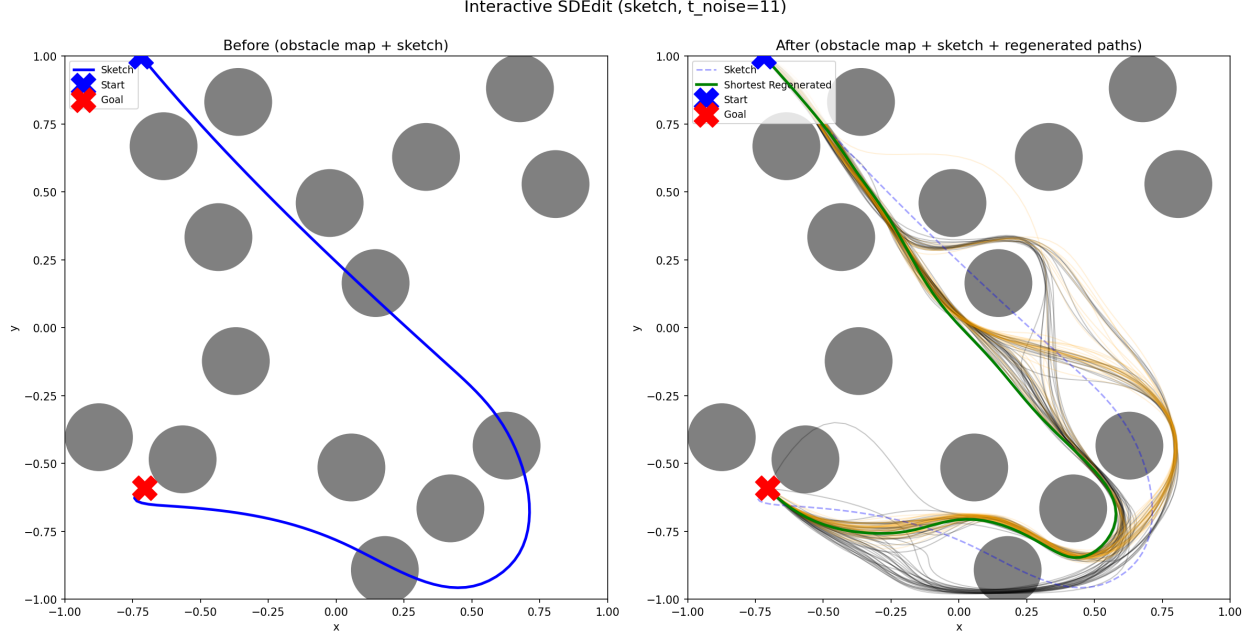


Figure 1: MPEdit capability: Sketch-To-Path. Blue line is the user sketch, Golden trajectories are valid trajectories, black are illegal trajectories, green is the shortest generated trajectory.

1. **Re-planning:** Given an existing trajectory in an existing obstacle map and a changed obstacle map (adding/removing obstacles to/from the existing obstacle map), MPEdit returns a legal, faithful (similar to original) trajectory for the changed obstacle map. It does that by adding noise to the existing trajectory at an intermediate diffusion time-step t_0 , and then denoises from t_0 to 0 with the collision-cost guide recomputed for the new environment. A larger t_0 gives the model more freedom to deviate from the input, trading faithfulness for a higher success rate. An example can be seen in figure 2.
2. **Sketch-to-path:** Users can draw approximate paths (potentially colliding with obstacles), which are then refined into valid, collision-free trajectories while preserving the sketch’s overall structure. The t_0 parameter has the same trade-off here. An example can be seen in figure 1.

Interactive GUI tools are provided for both obstacle editing and path sketching.

This report evaluates MPEdit on five experiments covering noise-level trade-offs, baseline comparisons, sketch-mode quality, inference speed, and output diversity. The results show that MPEdit is able to produce realistic and faithful motion planning paths from various different guides, in 2D and 3D scenarios.

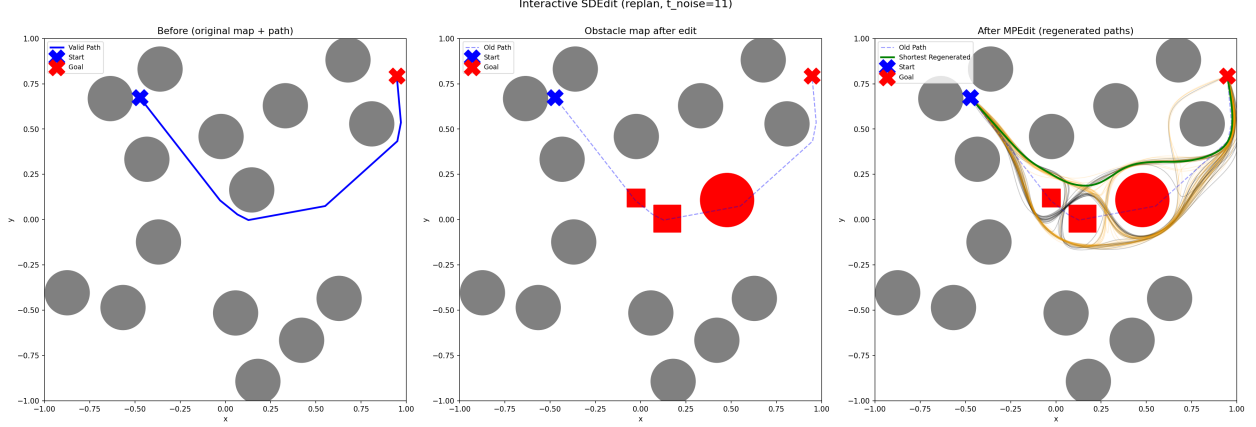


Figure 2: MPEdit capability: Re-planning.

2 Background

2.1 Motion Planning Diffusion (MPD)

Motion Planning Diffusion (MPD) formulates planning as posterior inference [1]:

$$p(\tau \mid O) \propto p(O \mid \tau) p(\tau), \quad (1)$$

where $\tau = (s_0, \dots, s_{H-1})$ is a trajectory, $p(\tau)$ is a learned trajectory prior, and $p(O \mid \tau)$ is a task likelihood that favors low-cost trajectories (collision-free, smooth, goal-reaching). Intuitively, MPD searches for trajectories that are both likely under the data distribution and likely to satisfy task objectives.

The diffusion forward process used to train the prior is

$$q(\tau_t \mid \tau_0) = \mathcal{N}(\tau_t; \sqrt{\bar{\alpha}_t} \tau_0, (1 - \bar{\alpha}_t)I), \quad (2)$$

which progressively corrupts clean trajectories τ_0 into noise. A denoiser then learns the reverse dynamics, giving a diffusion prior mean μ_t at each step during inference.

Task guidance is injected as a weighted gradient of planning costs:

$$g_t = - \sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1}) \Big|_{\tau_{t-1}=\mu_t}, \quad (3)$$

so each weight λ_i controls how strongly cost c_i (e.g., obstacle, smoothness, limits) shapes the update. In the implementation, these guide gradients are clipped/scaled before being combined with the diffusion prior.

This yields the practical MPD sampling step

$$\tau_{t-1} = \mu_t + g_t + \sigma_t z, \quad z \sim \mathcal{N}(0, I), \quad (4)$$

which moves samples toward lower-cost regions while preserving stochastic exploration through $\sigma_t z$. Start and goal states are re-imposed at every denoising step as hard constraints.

For obstacle avoidance, MPD uses differentiable signed-distance costs. With K collision spheres at FK positions $x_j(q)$ and signed distance $d(x_j)$,

$$c_{\text{obs}}(q) = \frac{1}{K} \sum_{j=1}^K \max(0, \epsilon - d(x_j(q))), \quad (5)$$

so the penalty is zero when clearance is above ϵ , and positive near/inside obstacles. Gradients backpropagate through FK/Jacobians, allowing guidance to push trajectories away from collisions.

In MPEdit, the same MPD mechanism is applied in normalized B-spline control-point space. The guide combines collision-object, self-collision, joint-limit, and smoothness costs; denoising then follows DDIM/DDPM steps from a user-selected t_0 , after which trajectories are reconstructed and collision-filtered.

2.2 SDEdit

SDEdit introduces stochastic differential equation (SDE)-based editing for guided image synthesis and manipulation [5]. Instead of denoising from pure Gaussian noise, the algorithm initializes the generative process using a user-provided guide $x^{(g)}$ (e.g., hand-drawn colored strokes or manipulated RGB pixels). SDEdit adds Gaussian noise of variance $\sigma^2(t_0)$ to the guide at an intermediate timestep $t_0 \in (0, 1)$, and subsequently simulates the reverse SDE from t_0 to 0 to produce the final synthesized output.

The initialization time t_0 explicitly regulates the inherent trade-off between realism and faithfulness to the input. A smaller t_0 injects minimal noise, forcing the reverse SDE to remain faithful to the original guide, which may compromise realism if the guide is out-of-distribution. Conversely, a larger t_0 smooths out undesirable artifacts and distortions by projecting the intermediate state further into the learned data distribution, increasing realism while decreasing adherence to the specific user input. Empirical findings suggest $t_0 \in [0.3, 0.6]$ achieves an optimal balance for standard editing tasks [5].

This framework provides a unified, zero-shot mechanism for various applications, including stroke-based image synthesis, image compositing, and targeted editing. It eliminates the need for task-specific dataset collection, conditional model retraining, and complex GAN inversion optimization routines required by previous methodologies [5].

3 Method

3.1 MPEdit - SDEdit for Motion Planning

Given an existing collision-free trajectory τ_{ref} (e.g., planned by RRT in the *original* obstacle map) and an updated obstacle map, MPEdit proceeds as follows:

1. **Convert to control points.** Map τ_{ref} to normalized B-spline control points $\mathbf{c}_0 \in \mathbb{R}^{K \times d}$ using least-squares fitting and the dataset normalization statistics.

Algorithm 1 MPedit: SDEdit-style path regeneration

Require: Reference path τ_{ref} , noise level t_0 , modified environment SDF, number of samples N

```
1:  $\mathbf{c}_0 \leftarrow \text{PathToCtrlPts}(\tau_{\text{ref}})$ 
2:  $\mathbf{c}_0 \leftarrow \text{Normalize}(\mathbf{c}_0)$ 
3: for  $i = 1$  to  $N$  do
4:    $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ 
5:    $\mathbf{c}_{t_0}^{(i)} \leftarrow \sqrt{\bar{\alpha}_{t_0}} \mathbf{c}_0 + \sqrt{1 - \bar{\alpha}_{t_0}} \epsilon$ 
6: end for
7: for  $t = t_0, t_0-1, \dots, 1$  do
8:    $\hat{\epsilon} \leftarrow \epsilon_\theta(\mathbf{c}_t^{(1:N)}, t, \mathbf{z})$   $\triangleright$  batched
9:    $\mathbf{c}_{t-1}^{(1:N)} \leftarrow \text{DDIMStep}(\mathbf{c}_t^{(1:N)}, \hat{\epsilon}) - \eta \nabla_{\mathbf{c}_t} \mathcal{J}_{\text{coll}}$ 
10:  Clamp start/goal control points
11: end for
12:  $\mathbf{c}_0^{(1:N)} \leftarrow \text{Unnormalize}(\mathbf{c}_0^{(1:N)})$ 
13:  $\mathcal{T} \leftarrow \{\tau^{(i)} \mid \tau^{(i)} = \text{BSpline}(\mathbf{c}_0^{(i)}), \text{ collision-free}\}$ 
14: return  $\arg \min_{\tau \in \mathcal{T}} \text{PathLength}(\tau)$ 
```

2. **Add noise.** Sample $\mathbf{c}_{t_0} \sim q(\mathbf{c}_{t_0} \mid \mathbf{c}_0)$ using (2) at a user-specified noise level t_0 .
3. **Denoise with updated cost guide.** Run DDIM reverse diffusion from step t_0 down to 0, with the collision-cost guide recomputed from the *modified* SDF. Hard boundary conditions are re-applied at every step.
4. **Select best path.** Among the N denoised trajectories, discard those that collide with the updated environment and select the shortest collision-free path.

The noise level t_0 is the key hyperparameter: it controls the trade-off between *faithfulness* to the reference path (measured by Fréchet and L2 distances) and the *success rate* (fraction of collision-free outputs). This report studies this trade-off systematically in Section 5.1.

Algorithm 1 summarizes the MPedit inference procedure.

3.2 Re-Planning Mode

In the re-planning scenario the environment changes after an initial path has been computed. Possible modifications include adding or removing obstacles. After the modification, the SDF is recomputed and the cost guide is rebuilt. MPedit then takes the reference path which might be invalid, adds noise, and denoises under the new cost, producing a trajectory that respects the updated map while staying close to the original route.

Obstacle modifications are classified into four difficulty levels: **easy** (small obstacle far from the path), **medium** (moderate obstacle near the path), **hard** (large obstacle blocking the path), and **removal** (an existing obstacle is removed).

3.3 Sketch-to-Path Mode

In sketch mode the user draws a rough path from start to goal using an interactive GUI (see Appendix B). The sketch is resampled to the model’s waypoint count via cubic B-spline fitting and then treated exactly like a reference path: noise is added at level t_0 and the model denoises it into a feasible trajectory. The trade-off of faithfulness and feasibility is controlled by both t_0 and the “corruption level” σ that measures how far the sketch deviates from a legal path.

4 Experimental Setup

Environments. The experiments run on two environments, with the pre-trained MPD models:

- **EnvSimple2D–PointMass2D:** a 2-D workspace with circular obstacles and a point-mass robot ($d=2$).
- **EnvSpheres3D–Panda:** a 3-D workspace with spherical obstacles and a 7-DOF Panda arm ($d=7$).

Baselines. Three methods are compared:

- **MPEdit (This project):** MPedit with noise level $t_0=11$ (tuned via Experiment 1).
- **Full MPD:** standard MPD inference from pure noise ($t_0=T$, i.e. the full diffusion schedule starting from random noise).
- **RRTConnect [3]:** classical bi-directional RRT.

Metrics.

- **Success rate (S%):** fraction of collision-free paths out of $N=100$ samples.
- **Discrete Fréchet distance:** maximum pointwise distance along the optimal alignment between the regenerated and input paths (lower = more faithful).
- **Mean L2 distance:** mean per-waypoint Euclidean distance after arc-length resampling.
- **Inference time:** wall-clock seconds on a single NVIDIA GPU.
- **Path length.**
- **Pairwise diversity:** mean pairwise Fréchet distance among all generated trajectories (higher = more diverse).

Protocol. Each experiment uses 50 start–goal pairs drawn from the validation set, with 100 trajectory samples per pair. All runs use seed 42 for reproducibility.

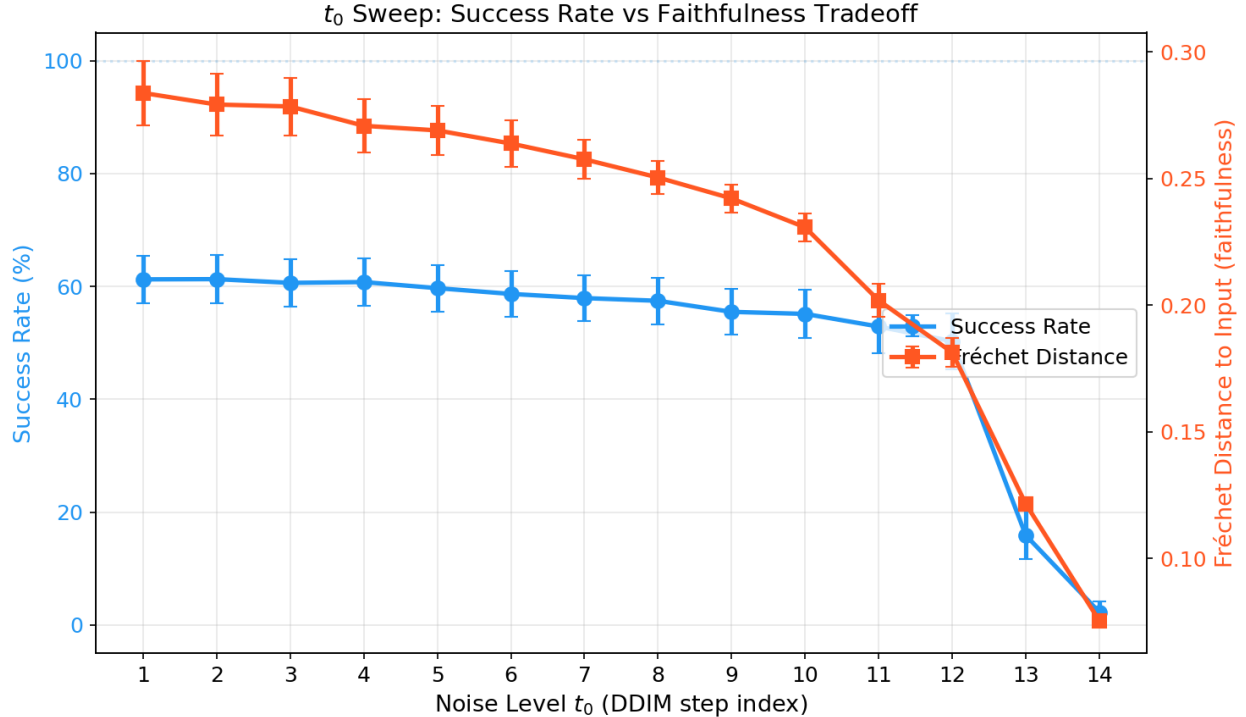


Figure 3: Noise-level trade-off. Success rate (left axis, %) decreases with t_0 , while faithfulness measured by Fréchet distance (right axis) increases (lower = more faithful).

5 Results

This section presents the primary 2-D quantitative results. The analogous 7-DOF Panda re-planning results in 3-D obstacle fields are reported in Appendix A, following the same flow (noise sweep, baseline comparison, inference speed, and diversity).

5.1 Experiment 1: Noise-Level Trade-Off (t_0 Sweep)

the noise level is swept $t_0 \in \{1, 2, \dots, 14\}$ on the medium difficulty scenario and plot success rate and Fréchet distance. Figure 3 shows the expected trade-off: low t_0 yields high distance (low faithfulness) but high success, meaning the trajectories diverge from the reference, but more of them are legal, while high t_0 increases faithfulness at the cost of lower success. The “elbow” around $t_0=11$ offers a good balance and is used in subsequent experiments. As mentioned in 2.2 this trade-off is also well documented in the SDEdit paper, although there is a slight difference between the t_0 values that reach ”optimal balance” in MPedit vs SDEdit.

5.2 Experiment 2: Baseline Comparison

Table 1 compares MPedit ($t_0=11$), Full MPD, and RRTConnect across the four difficulty scenarios.

Table 1: Re-planning comparison across difficulty scenarios (EnvSimple2D–PointMass2D, 50 pairs $\times$ 100 samples).

Scenario	Method	S%	Fréchet↓	L2↓	Time(s)↓	PathLen
easy	MPEdit	95.4%	0.067	0.036	0.468	1.799
easy	Full MPD	90.8%	0.199	0.094	0.770	1.825
easy	RRTConnect	100.0%	0.197	0.095	0.285	1.800
medium	MPEdit	38.7%	0.201	0.078	0.400	1.903
medium	Full MPD	57.6%	0.293	0.129	0.676	1.920
medium	RRTConnect	100.0%	0.391	0.197	0.395	1.949
hard	MPEdit	19.0%	0.241	0.118	0.433	2.124
hard	Full MPD	33.4%	0.337	0.174	0.694	2.113
hard	RRTConnect	100.0%	0.513	0.302	0.494	2.177
removal	MPEdit	99.5%	0.060	0.034	0.404	1.767
removal	Full MPD	97.7%	0.213	0.102	0.679	1.806
removal	RRTConnect	100.0%	0.164	0.068	0.237	1.765

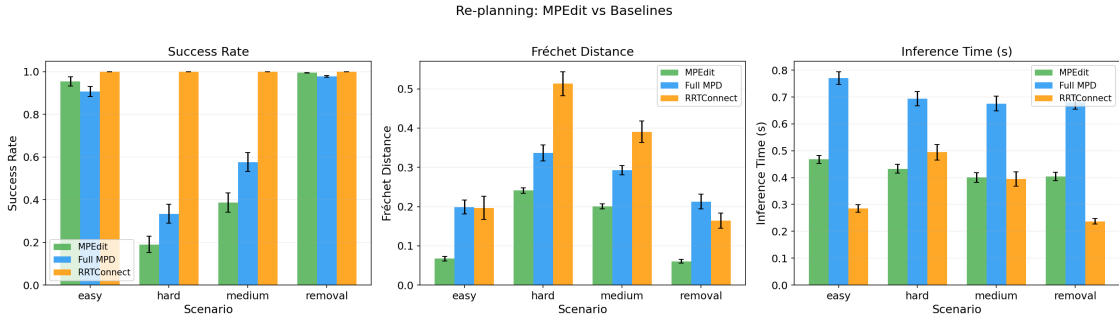


Figure 4: Performance comparison between different baselines in various scenarios

Key observations. MPEdit is strongest on *faithfulness*: it achieves the lowest Fréchet and L2 distances in every scenario (e.g., easy: 0.067/0.036 vs. 0.199/0.094 for Full MPD; removal: 0.060/0.034 vs. 0.164/0.068 for RRTConnect), confirming that initialization from the reference path preserves structure. In easy and removal, MPEdit also attains very high success (95.4% and 99.5%), close to RRTConnect and higher than Full MPD. However, under larger map changes (medium/hard), MPEdit success drops (38.7%, 19.0%), and Full MPD becomes more robust than MPEdit (57.6%, 33.4%). RRTConnect remains the feasibility leader (100% in all four scenarios), but this comes with substantially worse path similarity in medium/hard (Fréchet 0.391/0.513). Runtime-wise, MPEdit is consistently faster than Full MPD and is competitive with RRTConnect (slightly slower in easy/removal, near-equal in medium, faster in hard).

Figure 4 visualizes the success, distance and inference times as a grouped bar chart.

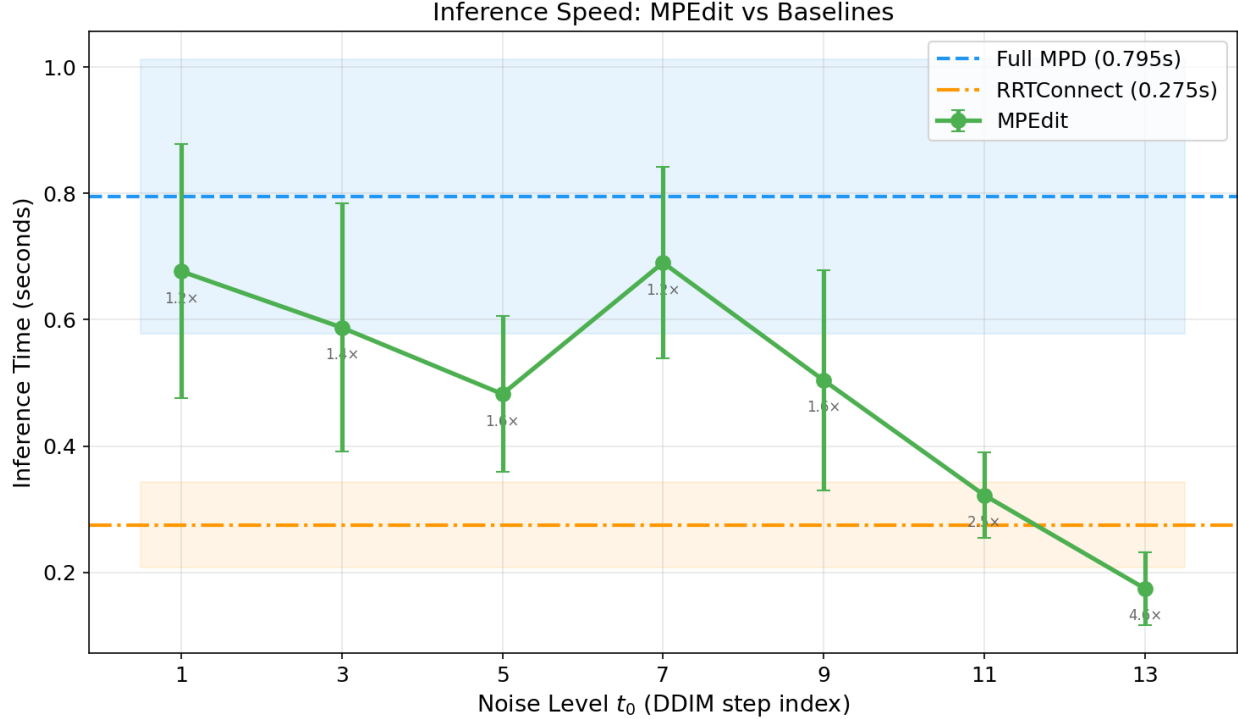


Figure 5: Inference speed vs. noise level.

5.3 Experiment 3: Inference Speed

Figure 5 reports wall-clock inference time as a function of t_0 . Because MPedit denoises from t_0 rather than the full schedule, lower noise levels yield proportionally faster inference. MPedit is significantly faster than full MPD and is competitive with RRTConnect on higher t_0 values.

5.4 Experiment 4: Output Diversity

This experiment explores diversity of the output. Figure 6 shows the mean pairwise Fréchet distance among the 100 generated trajectories per start-goal pair. Lower noise levels produce more diverse outputs; at $t_0=13$ the outputs are nearly identical to the input, while at $t_0=1$ diversity approaches that of Full MPD.

6 Discussion and Conclusion

Across both evaluated settings (2-D point-mass and 7-DOF Panda), MPedit’s most consistent advantage is *path faithfulness*: in the baseline tables, it achieves lower Fréchet and L2 distances to the reference trajectory than both Full MPD and RRTConnect. This supports the core claim of the method: editing from an existing path preserves trajectory intent better than regenerating from scratch.

Success-rate behavior depends on scenario difficulty. For moderate/local map edits, MPedit

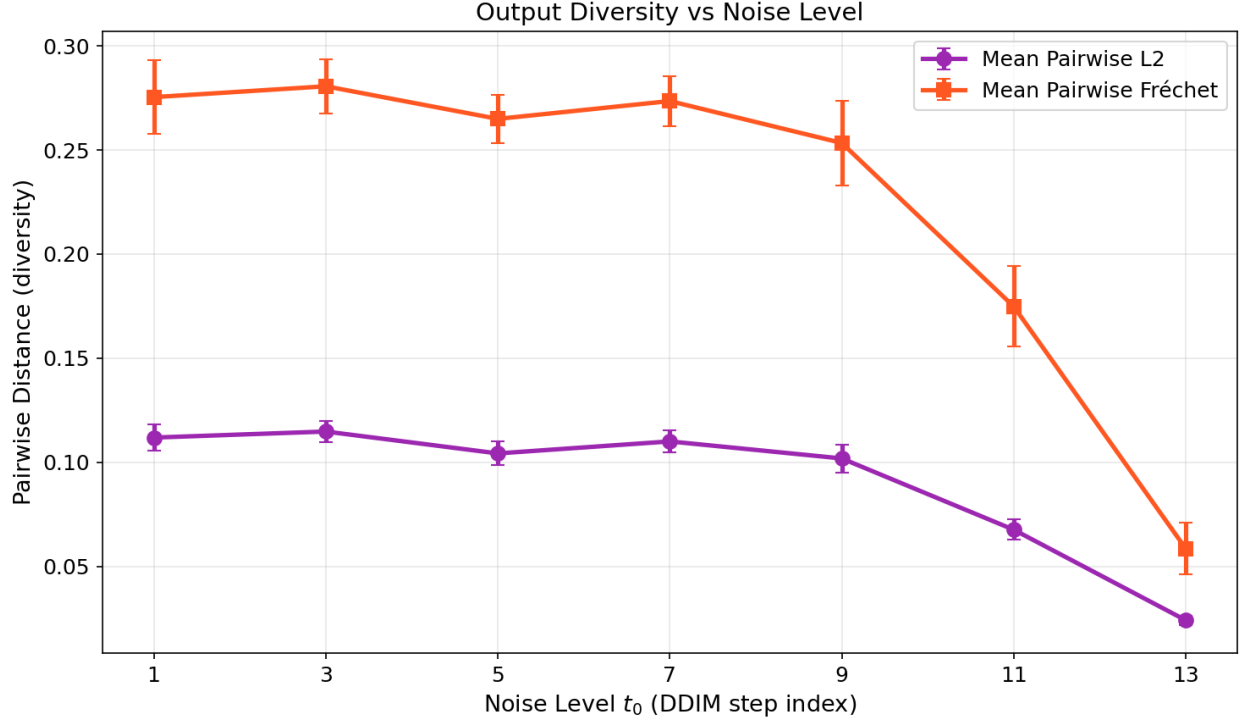


Figure 6: Output diversity vs. noise level.

remains competitive and often better than Full MPD (e.g., easy and removal in 2-D; easy/medium/removal in 3-D). For larger topological changes, robustness drops: in the hard scenario, Full MPD slightly outperforms MPedit in both 2-D and 3-D, while RRTConnect remains the feasibility leader. This indicates that strong adherence to the reference can become a liability when a major reroute is required.

The noise level t_0 is the main control knob behind this behavior. In both environments, t_0 governs the trade-off among faithfulness, success, and diversity: higher t_0 increases adherence to the input and lowers output spread, while lower t_0 allows larger edits and greater diversity. The reported sweeps suggest that intermediate values (around $t_0 \approx 11$ in the current setup) provide the best practical balance. In a different experimental setup, for example one that contains much harder obstacle maps would probably change our chosen t_0 value.

Runtime results show the expected efficiency benefit over Full MPD by skipping early denoising steps, while still generally trailing RRTConnect in absolute time. Taken together, these findings position MPedit as a practical “edit-first” replanning tool: it is especially useful when preserving the structure of an existing plan matters, and can be paired with a fallback planner (e.g., Full MPD or RRTConnect) when large environment changes demand more aggressive global rerouting.

References

- [1] João Carvalho, An T. Le, Mark Baierl, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and planning of robot motions with diffusion models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1916–1923, 2023. doi: 10.1109/IROS55552.2023.10342382. URL <https://doi.org/10.1109/IROS55552.2023.10342382>.
- [2] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2022. URL <https://arxiv.org/abs/2205.09991>.
- [3] James J. Kuffner and Steven M. LaValle. Rrt-connect: An efficient approach to single-query path planning. In *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation*, pages 995–1001, 2000. doi: 10.1109/ROBOT.2000.844730. URL <https://doi.org/10.1109/ROBOT.2000.844730>.
- [4] Steven M. LaValle. Rapidly-exploring random trees: A new tool for path planning. Technical Report TR 98-11, Computer Science Department, Iowa State University, 1998. URL <https://msl.cs.uiuc.edu/~lavalle/papers/Lav98c.pdf>.
- [5] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=aBsCjcPu_tE.

A 7-DOF Panda Re-Planning Results (3-D Obstacles)

This appendix mirrors Section 5 for the EnvSpheres3D-Panda setting. Planning is performed in 7-D joint space, while 3-D visualizations (for manual re-planning via GUI) are shown in task space after forward kinematics. Sketch mode is omitted here because freehand sketching isn’t very applicable in a 3d scenario.

A.1 Experiment A1: Noise-Level Trade-Off (t_0 Sweep)

As in the 2-D case, the noise level is swept $t_0 \in \{1, \dots, 14\}$ and measure success rate and faithfulness to the reference trajectory on the medium scenario. In 3-D, faithfulness (Fréchet/L2) improves monotonically as t_0 increases, while success is non-monotonic and peaks around $t_0 \in \{11, 12\}$ before dropping again at very high t_0 . This is probably due to the fact that the available environments are much more spacious, so there is relatively good success for all t_0 values.

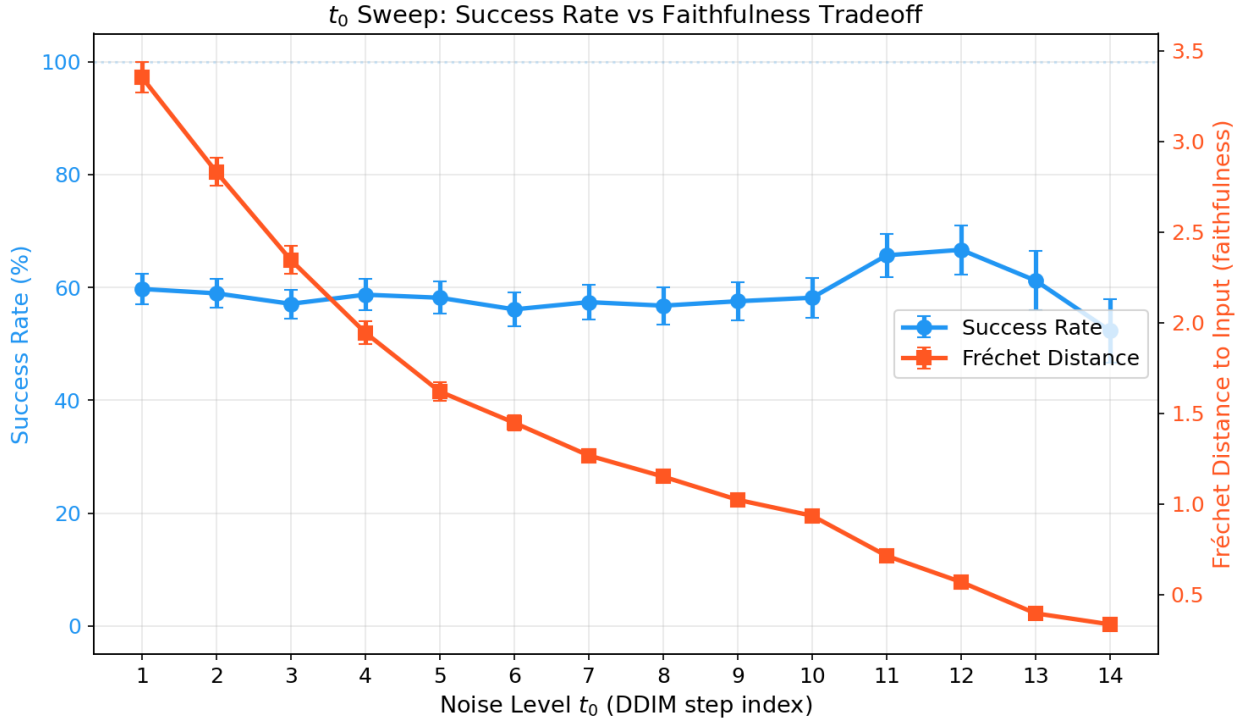


Figure 7: 7-DOF Panda: noise-level trade-off in 3-D obstacles.

A.2 Experiment A2: Baseline Comparison

Table 2 reports the 3-D baseline comparison between MPedit, Full MPD, and RRTConnect across the same four scenario types used in Section 5.2.

Table 2: Re-planning comparison across difficulty scenarios (EnvSpheres3D–Panda, 50 pairs $\times$ 100 samples).

Scenario	Method	S%	Fréchet↓	L2↓	Time(s)↓	PathLen
easy	MPedit	72.1%	0.606	0.347	3.294	7.714
easy	Full MPD	60.3%	4.327	2.252	4.312	8.379
easy	RRTConnect	100.0%	2.032	1.154	1.294	7.884
medium	MPedit	64.3%	0.706	0.387	3.219	8.276
medium	Full MPD	60.4%	4.259	2.186	4.174	8.371
medium	RRTConnect	100.0%	2.141	1.225	1.325	8.088
hard	MPedit	48.7%	0.769	0.441	4.139	8.444
hard	Full MPD	53.7%	4.272	2.227	5.415	8.548
hard	RRTConnect	100.0%	2.129	1.192	1.506	8.333
removal	MPedit	80.8%	0.529	0.307	4.797	7.560
removal	Full MPD	66.9%	4.279	2.198	6.459	8.172
removal	RRTConnect	100.0%	1.802	1.032	1.117	7.401

Key observations. MPedit is again strongest (by far) on faithfulness, with lower Fréchet/L2 than both Full MPD and RRTConnect in all scenarios. In success rate, MPedit outperforms Full MPD in easy/medium/removal, while Full MPD is slightly more robust in the hard scenario. RRTConnect reaches 100% success throughout and is fastest, but with substantially larger deviation from the reference trajectory.

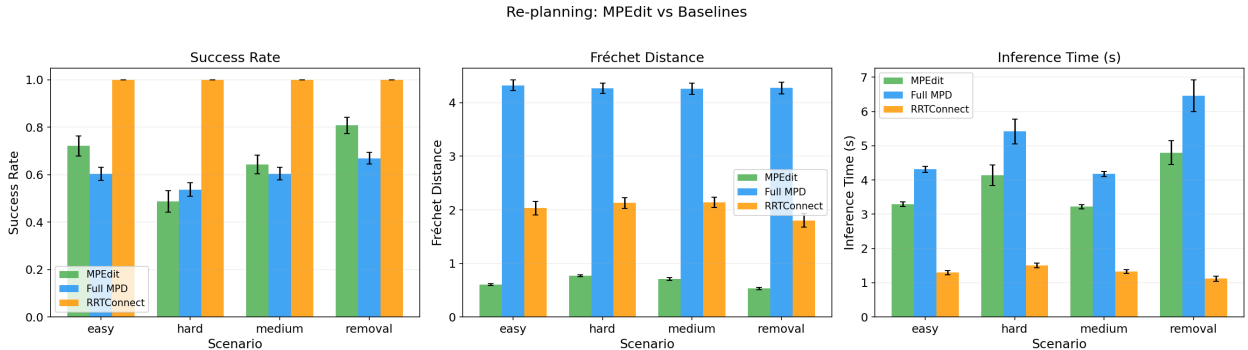


Figure 8: 7-DOF Panda: baseline comparison visualization across scenarios.

A.3 Experiment A3: Inference Speed

Inference latency is evaluated versus t_0 in the 7-DOF setting and compare against Full MPD and RRTConnect as in Section 5.3. The fastest SDEdit configuration occurs at high t_0 (e.g., $t_0=13$: ~ 1.67 s), while Full MPD averages ~ 4.30 s and RRTConnect remains faster (~ 0.67 s).

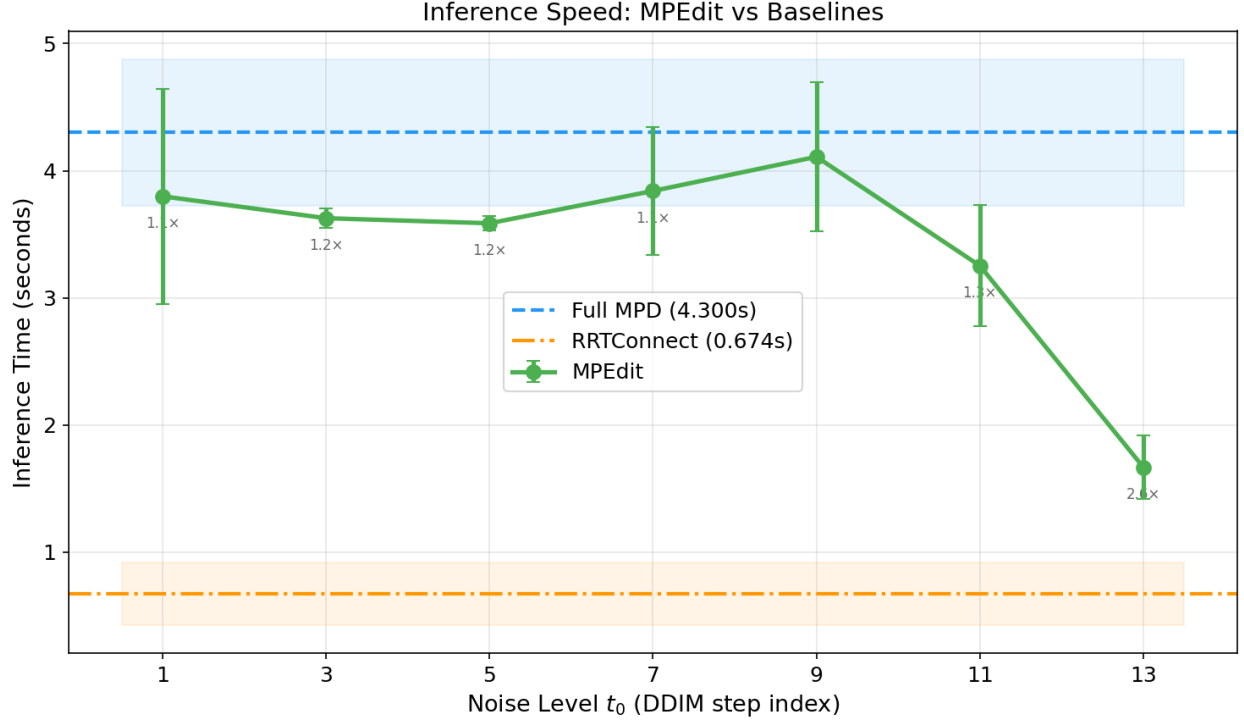


Figure 9: 7-DOF Panda: inference speed versus noise level.

A.4 Experiment A4: Output Diversity

Following Section 5.4, pairwise trajectory diversity is reported across noise levels for the Panda arm. As expected, diversity decreases with increasing t_0 (mean pairwise Fréchet from ~ 3.67 at $t_0=1$ to ~ 0.31 at $t_0=13$), showing progressively stronger adherence to the input path at higher t_0 .

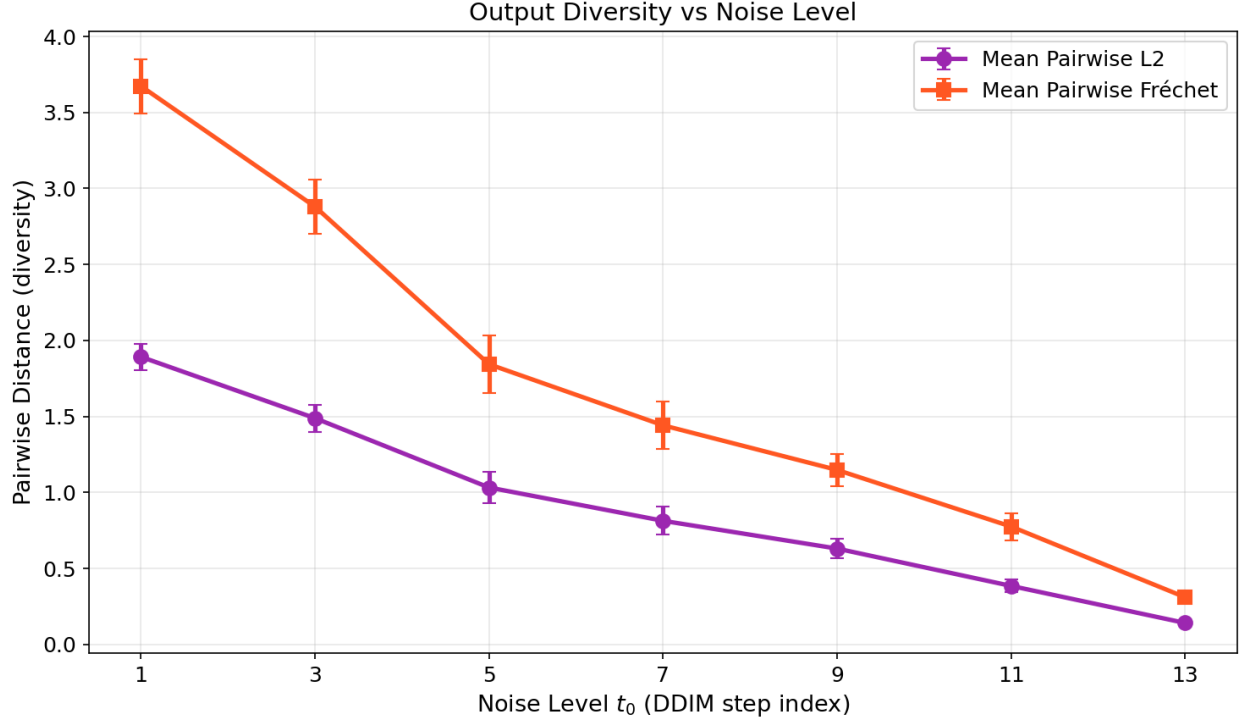


Figure 10: 7-DOF Panda: output diversity versus noise level.

B Sketch-to-Path Mode

The sketch-to-path mode allows a user to provide a rough, hand-drawn trajectory that is refined by the diffusion model into a collision-free, smooth path. This appendix describes the interactive tool and presents qualitative results.

B.1 Interactive Path Sketcher

The `PathSketcher` GUI (Figure 11) renders the 2-D obstacle map and lets the user draw a freehand curve from start to goal. Upon mouse release the raw sketch is processed:

1. Near-duplicate points are removed.
2. A cubic B-spline is fitted with smoothing parameter $s = 0.001$.
3. The spline is resampled to T waypoints to match the model input.
4. Endpoints are snapped exactly to the start and goal configurations.

The resulting sketch is displayed in green; the user can clear and redraw before confirming.

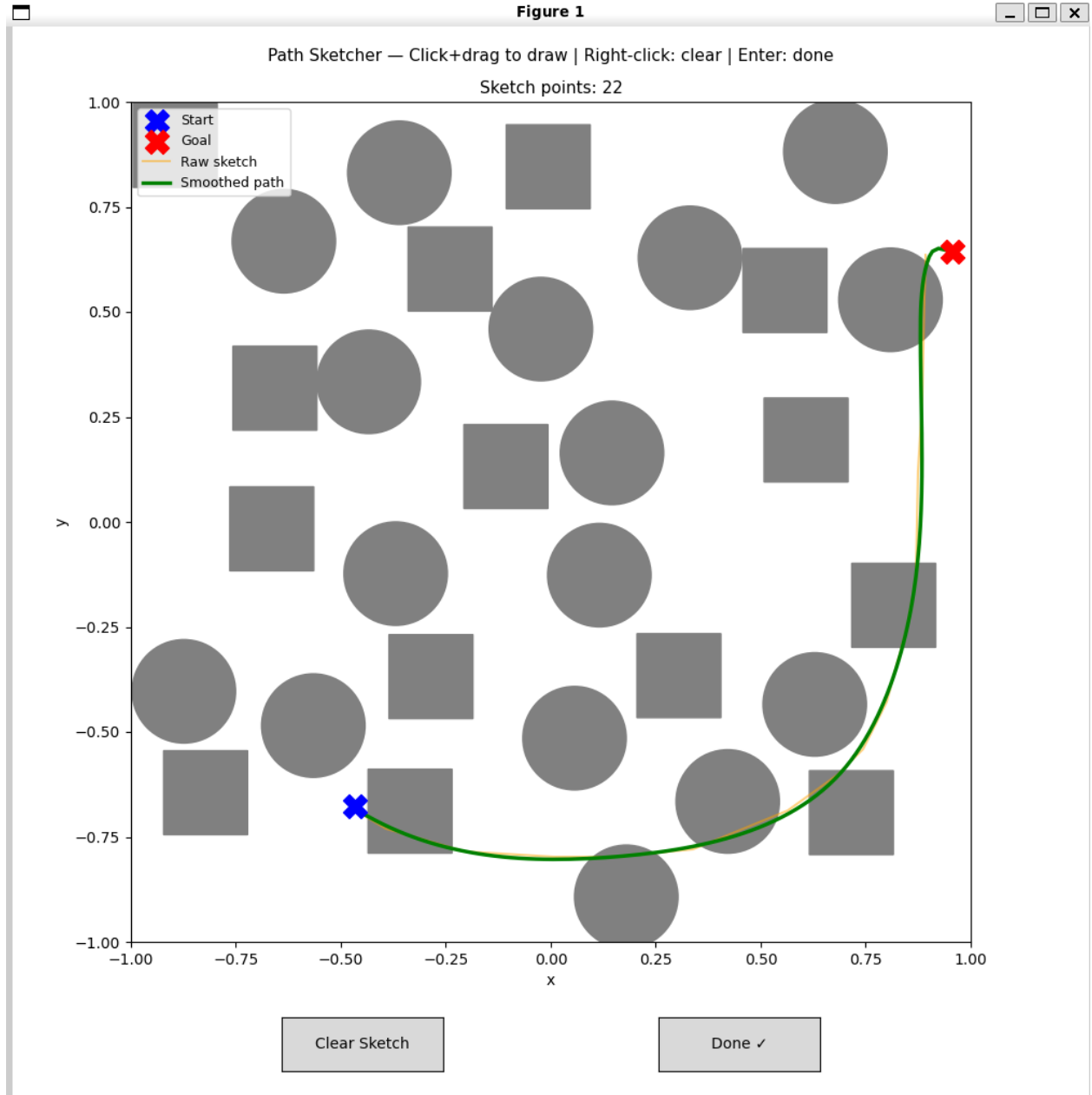


Figure 11: PathSketcher interactive GUI. The user draws a freehand path (orange) which is smoothed and resampled (green).

B.2 Sketch Experiment: Corruption vs. Noise Level

In the sketch experiment, two parameters are systematically varied:

- **Corruption level σ :** Gaussian noise added to a ground-truth path to simulate sketches of varying quality ($\sigma \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$).
- **Noise level t_0 :** the MPedit diffusion starting step ($t_0 \in \{1, 3, 5, 7, 9, 11, 13\}$).

Figure 12 shows the resulting heatmap of success rate and faithfulness (Fréchet distance) over the (σ, t_0) grid.

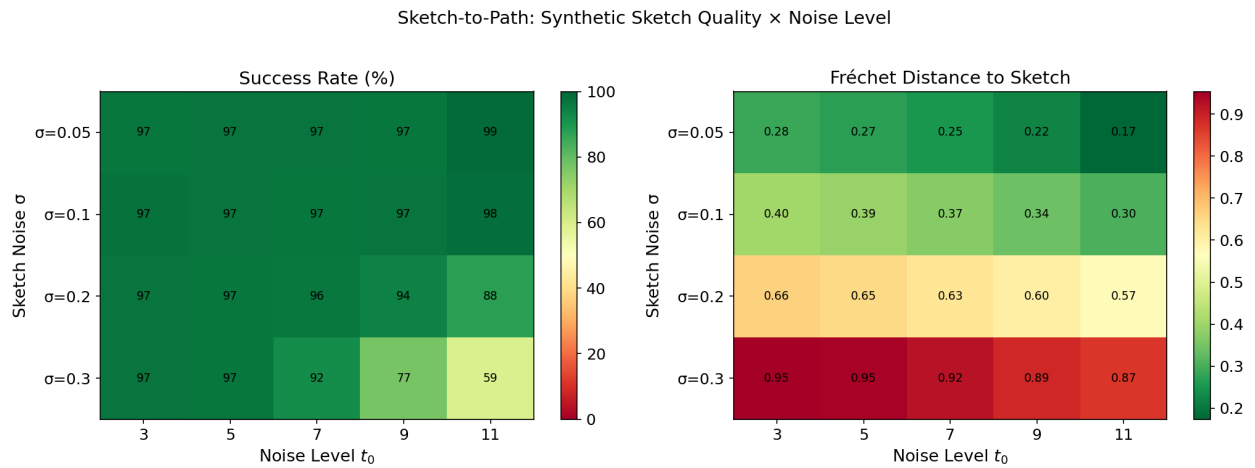


Figure 12: Sketch-mode heatmap. Each cell shows the success rate and mean Fréchet distance for a given (σ, t_0) combination.

B.3 Sketch-To-Path Examples

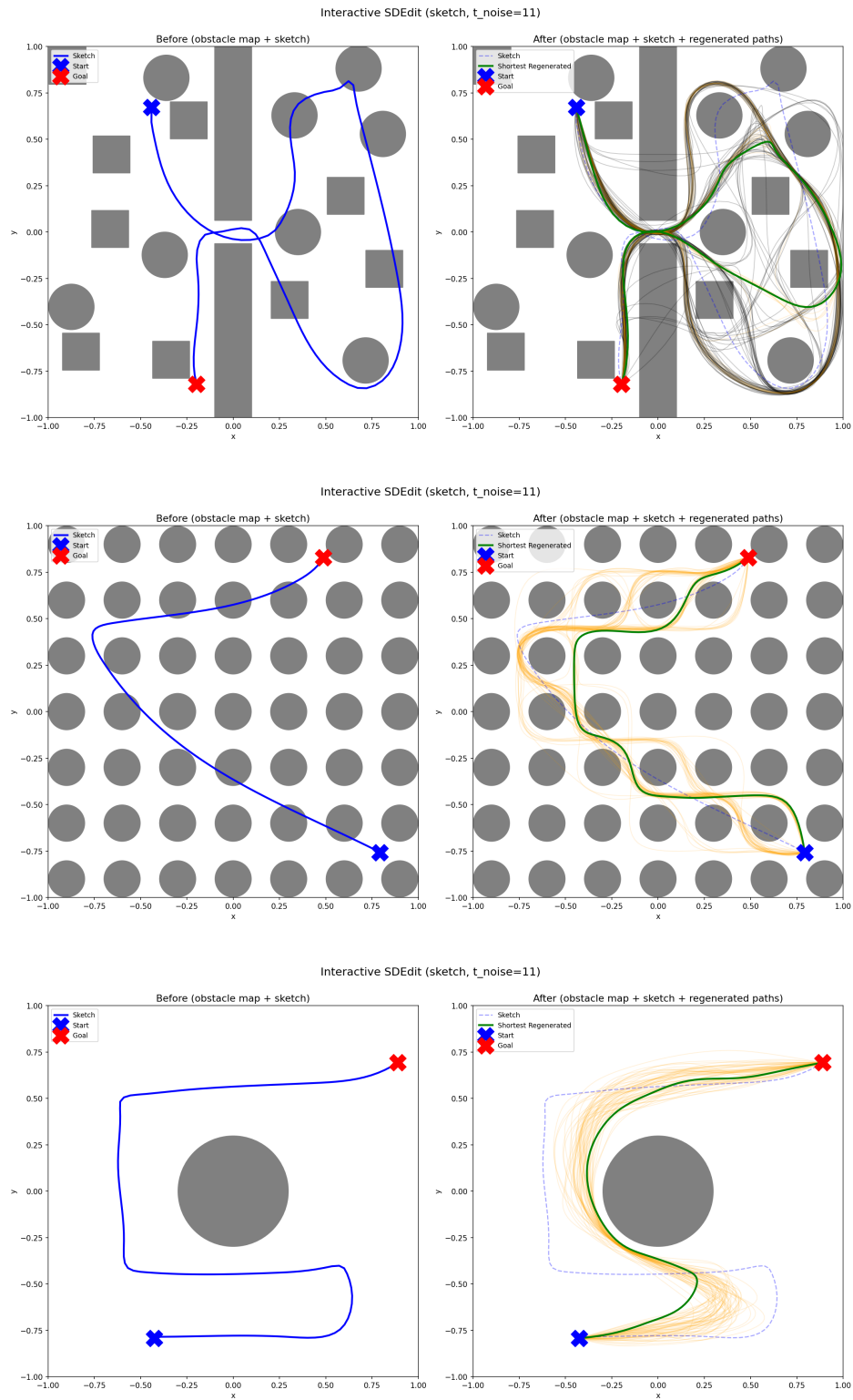


Figure 13: Sketch-to-path examples.

C Re-Planning Visualizations

The following images show the three-panel before/after visualization generated by MPedit's plotting utilities: the original environment with the reference path, the modified obstacle map, and the regenerated trajectories.

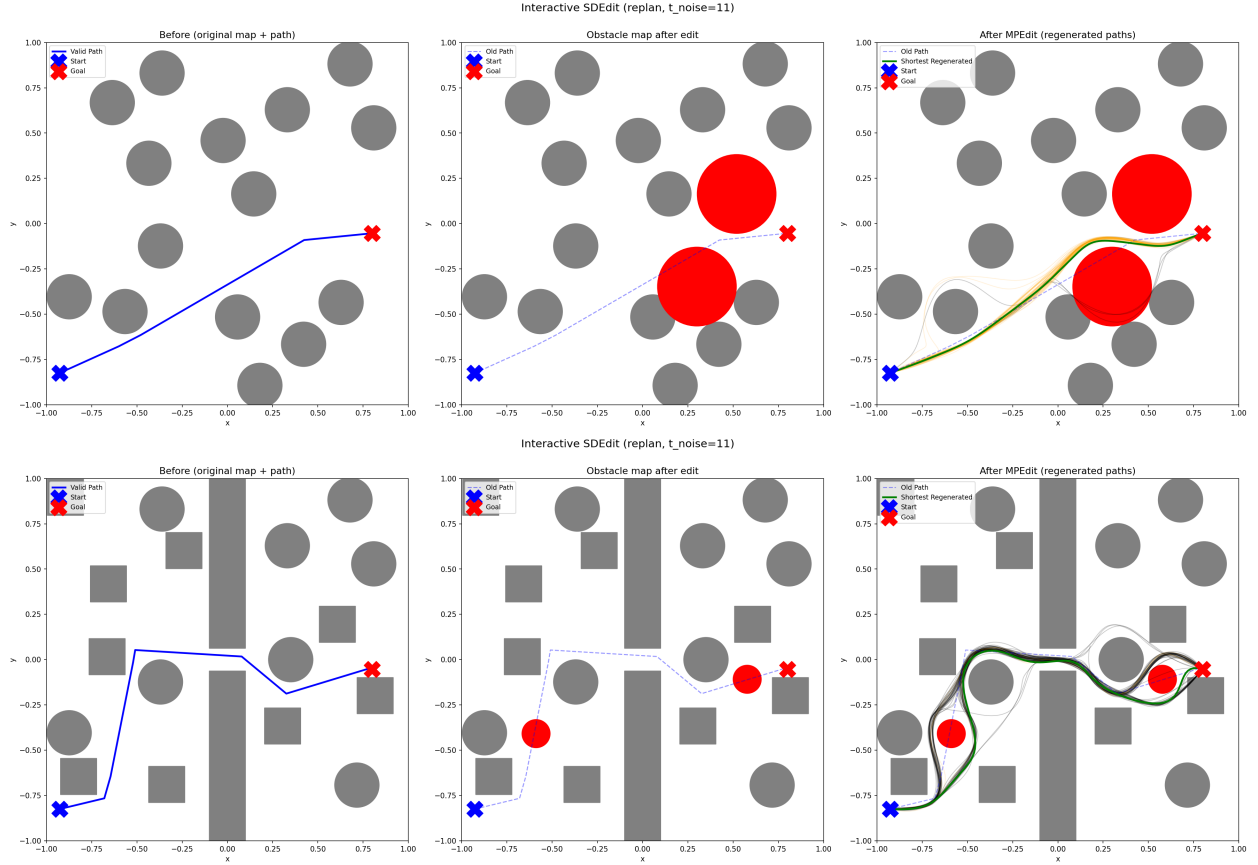


Figure 14: Re-planning visualization.